

JENKINS

THE COMPLETE GUIDE

Beginner to Advanced | Real Production Knowledge

Architecture | Multi-Agent | Scaling | Troubleshooting | Enterprise Patterns

BUILT FOR REAL INTERVIEWS AT REAL COMPANIES

Not tutorials. Not theory. How Jenkins actually works in production.

Every concept explained with: What it is | Why it exists | How it fails | How to fix it

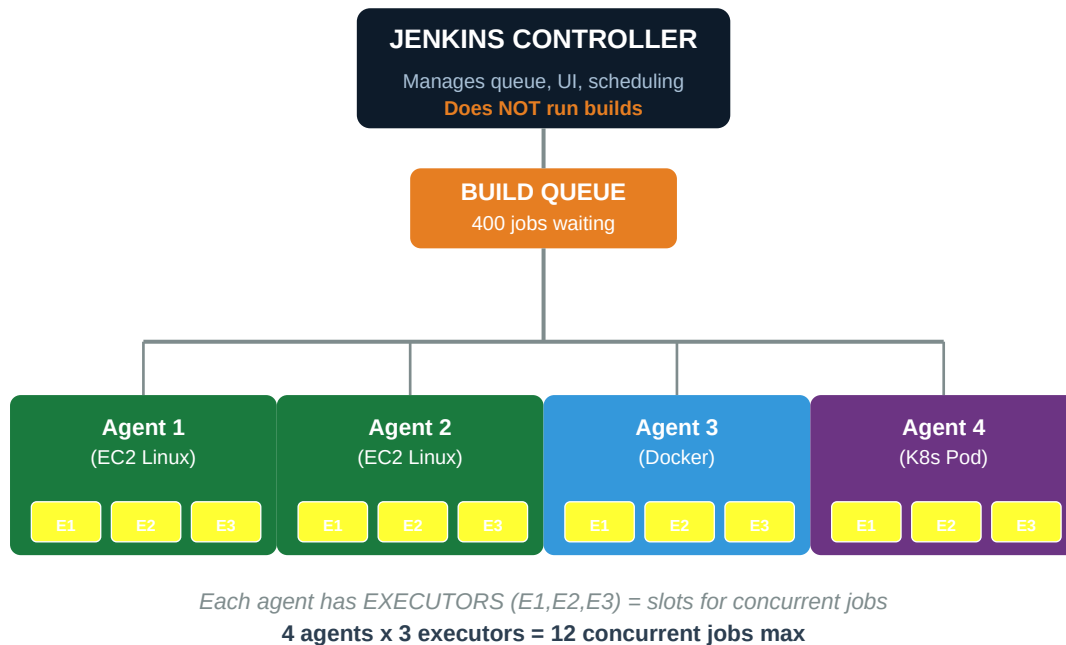
PART 1

How Jenkins Actually Works

Jenkins Architecture - The Mental Model

This is what failed you in the interview. Fix it now.

Jenkins is a Java application that runs on a server. It has two types of components: one Controller (brain) and one or more Agents (hands). Understanding this split is the single most important Jenkins concept.



The Three Core Components

Component	What It Is	Real-World Analogy	Key Fact
Controller (Master)	The main Jenkins server. Manages the UI, job queue, scheduling, build history, plugins.	Restaurant manager who takes orders and assigns tables	NEVER runs builds. If it dies, all agents are orphaned.
Agent (Node/Slave)	A worker machine that runs actual build commands. Can be EC2, Docker, K8s pod.	Kitchen where cooking happens. Multiple kitchens for more capacity.	Each agent has limited capacity (executors). More agents = more parallelism.
Executor	A slot on an agent. One executor = one concurrent build.	One chef in the kitchen. 3 executors = 3 chefs cooking simultaneously.	Set based on agent resources: 4GB RAM / 1GB per build = ~4 executors max.

The Build Queue - Where the 400 Jobs Problem Happens

When a build is triggered (by webhook, schedule, or manual click), it enters the BUILD QUEUE on the controller. The controller looks for a free executor on any available agent. If no executors are free, the job waits

in the queue. This is exactly what the interviewer described: 400 jobs waiting, 10 agents busy.

The bottleneck formula:

Max concurrent builds = Number of agents x Executors per agent 10 agents x 2 executors = 20 concurrent builds 400 jobs / 20 slots = 20 batches If each build takes 10 min = 200 min (3.3 hours) to drain the queue

THE KEY INSIGHT: This formula is the foundation of every Jenkins scaling discussion. When someone asks "how do you speed up Jenkins?", the answer is always about increasing the LEFT side (agents x executors) or decreasing the RIGHT side (build time per job).

What the Controller Stores (Critical for Backup)

Path	What It Contains	Why It Matters
/var/lib/jenkins/	Jenkins home directory. EVERYTHING is here.	This is what you back up.
/var/lib/jenkins/jobs/	Job configs + build history for every job	Losing this = losing all job history
/var/lib/jenkins/workspace/	Cloned code + build outputs per job	Recreatable - not critical to back up
/var/lib/jenkins/plugins/	All installed plugins	Critical - wrong plugin versions break Jenkins
/var/lib/jenkins/secrets/	Encrypted credentials (passwords, tokens, keys)	Most sensitive. Encrypt backups.
/var/lib/jenkins/nodes/	Agent configurations	How agents connect to controller
/var/lib/jenkins/config.xml	Global Jenkins configuration	Security settings, tool configs
/var/log/jenkins/jenkins.log	Jenkins application log	First place to look when Jenkins itself breaks

What Agents Store

Agents are disposable by design. They store temporary build workspaces and tool caches. If an agent dies, you lose nothing permanent - builds just get reassigned to other agents. This disposability is what makes dynamic (auto-scaling) agents possible.

Multi-Agent Architecture

The exact topic that failed you - now mastered

Real companies never run Jenkins with just one agent. A bank like Citizens Bank has 50-200 agents handling thousands of builds per day. Understanding multi-agent setup is essential.

4 Types of Jenkins Agents

Static Permanent Agents (EC2/VM)

What: A fixed set of machines always connected to Jenkins.

Setup: Launch EC2 instances, install Java, register in Jenkins as nodes.

Pros: Simple, predictable, always available.

Cons: Waste money when idle. Fixed capacity. Cannot scale up during peak.

Used at: Small teams, legacy setups, companies starting their Jenkins journey.

Configuration:

Jenkins -> Manage Jenkins -> Manage Nodes -> New Node

Name: build-agent-01

Type: Permanent Agent

Remote root: /home/jenkins

Launch method: Launch via SSH

Host: 10.0.1.50 (private IP of EC2)

Credentials: jenkins-ssh-key

Number of executors: 4

Labels: linux java maven docker

Docker Agents

What: Jenkins creates a Docker container as an agent for each build. Container destroyed after.

Setup: Install Docker on Jenkins server. Use Docker Pipeline plugin.

Pros: Clean environment every build. No "works on my agent" issues.

Cons: Requires Docker installed. Slightly slower startup than static agents.

Used at: Mid-size companies, teams wanting build isolation.

Jenkinsfile usage:

```
pipeline {
  agent {
    docker {
      image 'maven:3.9-openjdk-17'
      args '-v /root/.m2:/root/.m2' // cache Maven deps
    }
  }
  stages {
    stage('Build') { steps { sh 'mvn clean package' } }
  }
}
```

Kubernetes Pod Agents (THE ENTERPRISE ANSWER)

What: Jenkins creates a Kubernetes pod as an agent. Pod destroyed after build. Auto-scales.

Setup: Jenkins Kubernetes Plugin. Configure cloud pointing to K8s cluster.

Pros: Infinite scaling. No idle costs. Perfect isolation. THE enterprise solution.

Cons: Requires Kubernetes cluster. More complex setup.

Used at: Large enterprises, banks, tech companies. THIS IS WHAT CITIZENS BANK LIKELY USES.

Jenkinsfile usage:

```
pipeline {
  agent {
    kubernetes {
      yml """
      apiVersion: v1
      kind: Pod
      spec:
        containers:
        - name: maven
          image: maven:3.9-openjdk-17
          command: ['cat']
        - name: docker
          image: docker:latest
          command: ['cat']
        tty: true
      """
    }
  }
  stages {
    stage('Build') {
      steps {
        container('maven') { sh 'mvn clean package' }
      }
    }
    stage('Docker') {
      steps {
        container('docker') { sh 'docker build .' }
      }
    }
  }
}
```

EC2 Cloud Agents

What: Jenkins EC2 Plugin launches EC2 instances when queue builds up. Terminates when idle.

Setup: EC2 Plugin + AWS credentials + AMI with Java pre-installed.

Pros: Auto-scaling for non-Kubernetes environments. Cost-efficient.

Cons: EC2 boot time (~60-90 seconds). AMI maintenance required.

Used at: Companies using AWS but not yet on Kubernetes.

Configuration:

Jenkins -> Manage Jenkins -> Configure Clouds -> Amazon EC2

AMI: ami-xxxx (Ubuntu + Java 17 pre-installed)

Instance type: c5.xlarge

Min instances: 0

Max instances: 50

Idle termination: 15 minutes

Label: ec2-linux

The 400 Jobs Problem - Complete Answer

The exact question that failed you, answered completely

"How would you speed up the process if 400 jobs are in queue and 10 agents are running on Jenkins?"

4 Strategies to Speed Up Jenkins



Use all 4 together for maximum throughput

Strategy 1: Add More Agents (Immediate Relief)

The most direct fix. More agents = more parallel capacity. 10 agents cannot serve 400 jobs fast enough. 40 agents with 2 executors each = 80 concurrent builds.

```
# Add a new static agent Jenkins -> Manage Jenkins -> Manage Nodes -> New Node Name:
build-agent-11 Number of executors: 4 Labels: linux java maven Launch via SSH -> host:
10.0.1.55 # Or if using EC2 Plugin: increase max instances from 10 to 50 Jenkins -> Configure
Clouds -> EC2 -> Max instances: 50
```

Strategy 2: Increase Executors Per Agent

If your agents have spare CPU and RAM, increase executors. Each executor is one concurrent build slot.

```
# Check agent resources first ssh build-agent-01 free -h # how much RAM free? nproc # how many
CPU cores? df -h # how much disk free? # If agent has 8GB RAM and builds use ~1GB each: # Safe
to set 6-7 executors # Change in Jenkins: Jenkins -> Manage Nodes -> build-agent-01 ->
Configure Number of executors: 1 -> 6 (was 1, now 6) # Result: same 10 agents, 6x more capacity
# Before: 10 agents x 1 executor = 10 concurrent builds # After: 10 agents x 6 executors = 60
concurrent builds
```

CAUTION: Warning: setting executors too high causes OOM (Out of Memory) kills and build failures.
Rule of thumb: (Agent RAM - 1GB for OS) / (RAM per build) = max executors. Monitor after changing.

Strategy 3: Dynamic Cloud Agents (The Enterprise Answer)

Static agents sit idle when no builds are running. Dynamic agents are created on demand and destroyed when done. This is the correct production answer.

Type	How It Works	Startup Time	Cost Model
------	--------------	--------------	------------

Kubernetes Pods	K8s plugin creates a pod per build. Pod dies after.	~5 seconds	Pay only during build
EC2 Instances	EC2 plugin launches instances from AMI. Terminates after idle.	~60-90 seconds	Pay per minute of instance
Docker Containers	Docker plugin creates container per build.	~2 seconds	Free (runs on existing server)

With Kubernetes dynamic agents: - 400 jobs in queue -> K8s creates up to 400 pods (if cluster has capacity) - Each pod runs one build - Pods terminate after build completes - Queue drains in ~1 batch (the time of one build) - No idle agents wasting money between builds

Strategy 4: Optimize Individual Build Time

Shorter builds free agents faster = more throughput from the same agents.

Optimization	What It Does	Impact
Parallel stages	Run tests and scans simultaneously instead of sequentially	30-50% faster pipeline
Maven dependency caching	Cache ~/.m2/ on agents so dependencies are not re-downloaded	5-10 min saved per build
Docker layer caching	Reuse unchanged layers from previous builds	2-5 min saved per Docker build
Skip unnecessary stages	Feature branches skip deploy stages (only main deploys)	20-30% shorter for branches
Throttle duplicate builds	Cancel older builds when a newer push arrives for same branch	Reduces queue by 20-40%
Incremental builds	Only compile changed modules (Maven -pl flag)	50-70% faster for large projects

Parallel Stages Example:

```
stage('Quality Checks') { parallel { stage('Unit Tests') { steps { sh 'mvn test -Punit' } } stage('SonarQube') { steps { sh 'mvn sonar:sonar' } } stage('OWASP') { steps { sh 'mvn dependency-check:check' } } } } // All three run simultaneously instead of sequentially // 15 min (sequential) -> 5 min (parallel)
```

Strategy 5: Queue Management

Not all 400 jobs are equal. Prioritize and throttle.

Plugin	What It Does	When to Use
Priority Sorter	Assign priority to jobs. Production builds jump the queue.	Always. Production should never wait behind feature builds.
Throttle Concurrent Builds	Limit how many instances of the same job can run simultaneously.	When the same job triggers 10 times from rapid pushes.

Build Discarder	Auto-delete old build history to save disk and reduce Jenkins load.	Always. Keep last 30 builds, delete rest.
Quiet Period	Wait N seconds after trigger before starting (in case more commits come).	Feature branches with frequent pushes.

The Complete Answer to "400 Jobs, 10 Agents"

MEMORIZE THIS ANSWER

Immediate (5 minutes):

1. Check if any agents are offline - bring them back online
2. Check for stuck jobs eating executor slots - kill them
3. Increase executors on existing agents if they have spare resources

Short term (same day):

4. Add more static agents (register new EC2 instances)
5. Enable Priority Sorter plugin - production jobs first
6. Throttle duplicate builds on same branches

Proper fix (this sprint):

7. Implement dynamic agents (Kubernetes or EC2 plugin) that auto-scale with queue depth
8. Add parallel stages to pipelines to reduce individual build time
9. Cache Maven dependencies on agents
10. Skip unnecessary stages on feature branches

Why dynamic agents are the real answer:

With Kubernetes agents, 400 jobs = 400 pods created automatically. Queue drains in one build cycle. When queue is empty, pods terminate. Zero idle cost. This is what large enterprises use.

PART 2

Pipelines Deep Dive

Declarative vs Scripted Pipeline

Feature	Declarative (Recommended)	Scripted (Legacy)
Syntax	Structured blocks: pipeline{}, stages{}, stage{}	Free-form Groovy: node{}, stage{}
Readability	High - anyone can read it	Low - requires Groovy knowledge
Error handling	Built-in post{} block	Manual try/catch
When to use	99% of cases. Enterprise standard.	When Declarative cannot express your logic
Example start	pipeline { agent any }	node { stage("Build") { } }

Complete Declarative Jenkinsfile with Every Feature

```

pipeline { agent any // where to run options { timeout(time: 30, unit: 'MINUTES') // kill if
stuck > 30 min buildDiscarder(logRotator( numToKeepStr: '20')) // keep last 20 builds
disableConcurrentBuilds() // prevent parallel runs of same job timestamps() // add timestamps
to console } parameters { string(name: 'VERSION', defaultValue: 'latest', description: 'Version
to deploy') choice(name: 'ENV', choices: ['dev','qa','prod'], description: 'Target env')
booleanParam(name: 'SKIP_TESTS', defaultValue: false) } environment { APP_NAME =
'payment-service' NEXUS_URL = credentials('nexus-url') // from Jenkins credentials SONAR_TOKEN
= credentials('sonar-token') } tools { maven 'Maven-3.9' jdk 'JDK-17' } triggers { githubPush()
// trigger on GitHub webhook cron('H 2 * * 1-5') // also run at 2am weekdays } stages {
stage('Checkout') { steps { checkout scm echo "Branch: ${env.BRANCH_NAME}" echo "Build:
${env.BUILD_NUMBER}" } } stage('Build') { steps { sh 'mvn clean package -DskipTests' } }
stage('Tests') { when { expression { return !params.SKIP_TESTS } } steps { sh 'mvn test' } post
{ always { junit 'target/surefire-reports/*.xml' } } } stage('Quality Checks') { parallel { //
run in parallel stage('SonarQube') { steps { withSonarQubeEnv('SQ') { sh 'mvn sonar:sonar' } }
} stage('OWASP') { steps { sh 'mvn dependency-check:check -DfailBuildOnCVSS=7' } } }
stage('Quality Gate') { steps { timeout(time: 5, unit: 'MINUTES') { waitForQualityGate
abortPipeline: true } } } stage('Docker Build & Push') { steps { script { def img =
docker.build("${APP_NAME}:${BUILD_NUMBER}") docker.withRegistry('https://registry.company.io',
'reg-creds') { img.push() img.push('latest') } } } } stage('Deploy') { when { branch 'main' //
only deploy from main } input { message 'Deploy to production?' // manual approval gate ok
'Yes, deploy' submitter 'devops-team,managers' // only these users can approve } steps { sh ""
helm upgrade --install ${APP_NAME} ./helm \ --set image.tag=${BUILD_NUMBER} \ --namespace
production "" } } } post { always { cleanWs() // clean workspace } success { slackSend
channel: '#deploy', color: 'good', message: "OK: ${APP_NAME} #${BUILD_NUMBER}" } failure {
slackSend channel: '#deploy', color: 'danger', message: "FAIL: ${APP_NAME} #${BUILD_NUMBER}" }
} }

```

Every Block Explained

Block	Purpose	Key Details
pipeline {}	Top-level wrapper	Required. Everything goes inside.

agent	Where to run	any, none, docker{}, kubernetes{}, label
options {}	Job-level settings	timeout, buildDiscarder, timestamps
parameters {}	User inputs	string, choice, boolean. Accessed via params.NAME
environment {}	Variables	credentials() for secrets. Available in all stages.
tools {}	Tool installations	Must match names in Jenkins Global Tools
triggers {}	Auto-trigger rules	githubPush(), cron(), pollSCM()
stages {}	Pipeline phases	Sequential by default. Parallel inside a stage.
when {}	Conditional execution	branch, expression, environment, changeset
input {}	Manual approval	message, ok button text, submitter restrictions
parallel {}	Run stages simultaneously	Must be inside a stage. Waits for all to finish.
post {}	After-pipeline actions	always, success, failure, unstable, aborted
script {}	Escape to scripted pipeline	Use when Declarative cannot express your logic

ENTERPRISE PATTERN: The "input" block is how enterprises implement approval gates. At Citizens Bank, the Deploy stage would have an input requiring a manager from the release team to approve before production deployment. This is a compliance requirement.

PART 3

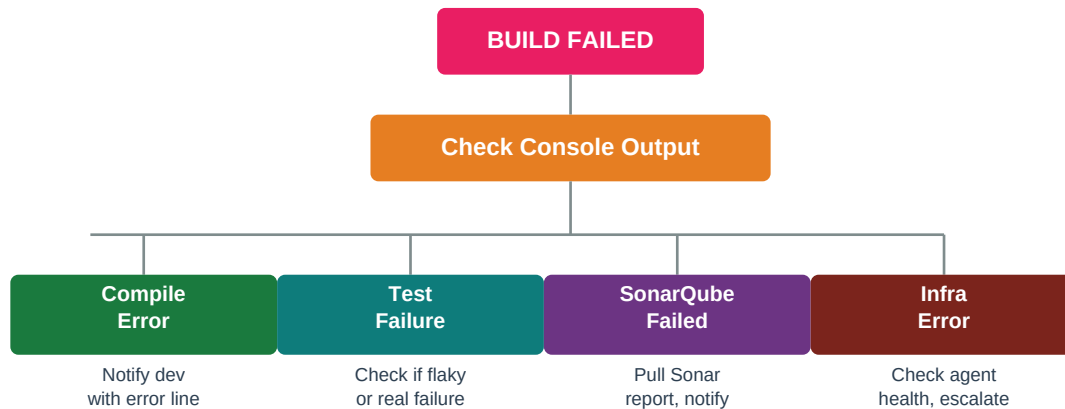
Real Production Scenarios

15 Production Scenarios You Will Face

How to diagnose and fix real Jenkins problems

These are the actual problems that L1 and L2 engineers deal with daily. Each scenario includes: what you see, what caused it, and exactly what you do.

Jenkins Troubleshooting Decision Tree



Scenario 1: Build Queue Backed Up (400 jobs)

Symptom: Jenkins dashboard shows hundreds of jobs queued. Developers complain builds take hours.

Cause: Insufficient agents/executors for the workload.

Your action:

1. Jenkins -> Build Executor Status -> check how many agents online vs offline
2. Are any builds stuck? Console Output shows no progress for 30+ min -> kill them
3. Offline agents? -> check SSH connectivity, disk space, Java process
4. All agents healthy + all slots full = genuine capacity issue -> escalate to L2
5. L2 will: increase executors, add agents, or check auto-scaling config

Prevention: Dynamic agents (K8s/EC2 plugin), Priority Sorter, throttle concurrent builds

Scenario 2: Agent Went Offline

Symptom: Jenkins -> Manage Nodes shows agent as offline (red X).

Cause: SSH connection dropped, agent EC2 stopped, Java process crashed, disk full.

Your action:

1. Click the offline agent -> read the "Offline reason" message
2. Common messages:
 - "Connection was broken" -> SSH issue. Try "Launch agent" button
 - "Disk space too low" -> agent disk full. SSH in, clean /tmp, Docker images, old builds
 - "java.lang.OutOfMemoryError" -> agent ran out of RAM. Restart agent process
3. SSH into the agent: `ssh jenkins@10.0.1.50`
 - Check: `df -h` (disk), `free -h` (memory), `ps aux | grep java` (process)
4. If EC2 is stopped: AWS Console -> start the instance

Prevention: Monitor agent health with Prometheus/Datadog. Alert when disk > 80%.

Scenario 3: Build Stuck - Not Progressing

Symptom: Build #347 has been "running" for 3 hours. Normal build time is 10 minutes.

Cause: Waiting for input, network timeout, deadlock, Docker daemon hung.

Your action:

1. Click build -> Console Output -> scroll to bottom
2. If last line is "Waiting for user input" -> someone needs to approve/reject
3. If last line is "Downloading..." for hours -> network issue or Maven Central slow
4. If output stopped completely -> build process is hung
5. Kill the build: click red X on the build page
6. If kill button doesn't work: Manage Jenkins -> Script Console -> run:
`Jenkins.instance.getItemByFullName("job-name").getBuildByNumber(347).doStop()`

Prevention: pipeline timeout option (`timeout(time: 30, unit: 'MINUTES')`)

Scenario 4: Jenkins Disk Full

Symptom: Builds fail with "No space left on device". Jenkins UI may become slow or unresponsive.

Cause: Build history, workspace files, Docker images accumulating over months.

Your action:

1. SSH into Jenkins controller: `df -h`
2. Find largest directories: `du -sh /var/lib/jenkins/* | sort -rh | head`
3. Common culprits:
 - `/var/lib/jenkins/workspace/` -> old workspaces. Delete unused: `rm -rf workspace/old-job*`
 - `/var/lib/jenkins/jobs/*/builds/` -> old build artifacts
 - Docker images: `docker system prune -a` (on agents)
4. Enable build discarder on ALL jobs: `numToKeepStr: '20'`
5. Configure Workspace Cleanup plugin for automatic cleanup

Prevention: Build discarder in every Jenkinsfile. Monitor disk with alerts at 80% threshold.

Scenario 5: Jenkins Out of Memory (OOM)

Symptom: Jenkins UI becomes extremely slow or crashes. `java.lang.OutOfMemoryError` in logs.

Cause: Too many jobs, too many plugins, too much build history loaded in memory.

Your action:

1. Restart Jenkins: `sudo systemctl restart jenkins`
2. If restart fixes it temporarily: increase heap size
Edit `/etc/default/jenkins`:
`JAVA_ARGS="-Xmx4g -Xms2g"` (set max heap to 4GB)
3. Reduce memory pressure: delete old build history, reduce kept builds
4. Audit plugins: Manage Jenkins -> Manage Plugins -> remove unused

Prevention: Monitor JVM heap with Prometheus JMX exporter. Alert at 80% heap usage.

Scenario 6: Plugin Conflict After Update

Symptom: Jenkins shows errors after plugin update. Some features broken.

Cause: New plugin version incompatible with Jenkins version or other plugins.

Your action:

1. Check `/var/log/jenkins/jenkins.log` for plugin-related errors
2. Downgrade the plugin: Manage Jenkins -> Manage Plugins -> Installed
-> Find plugin -> "Downgrade to X.Y.Z"
3. If Jenkins won't start: SSH in, remove the plugin from `/var/lib/jenkins/plugins/`
`rm /var/lib/jenkins/plugins/bad-plugin.jpi`
`rm -rf /var/lib/jenkins/plugins/bad-plugin/`
`sudo systemctl restart jenkins`

Prevention: NEVER update plugins on production Jenkins without testing first. Have a staging Jenkins.

Scenario 7: Credentials Expired

Symptom: All builds fail at "Push to Nexus" or "Docker Push" with 401 Unauthorized.

Cause: Nexus/Docker/GitHub token expired or password was rotated.

Your action:

1. Check console output: look for "401 Unauthorized" or "authentication failed"
2. Identify which credential is failing (Nexus, Docker, GitHub, SonarQube)
3. Get new credentials from the team that manages the tool
4. Update in Jenkins: Manage Jenkins -> Manage Credentials -> find the credential -> Update
5. Re-run a failed build to confirm fix

Prevention: Track credential expiry dates. Set calendar reminders 2 weeks before.

Scenario 8: Webhook Not Triggering Builds

Symptom: Developer pushes code but Jenkins doesn't start a build.

Cause: Webhook misconfigured, Jenkins URL changed, firewall blocking GitHub.

Your action:

1. GitHub repo -> Settings -> Webhooks -> check recent deliveries
 - Green check = GitHub sent successfully, Jenkins didn't process
 - Red X = GitHub couldn't reach Jenkins (URL wrong or firewall)
2. Verify webhook URL: `http://:8080/github-webhook/` (note trailing /)
3. Check Jenkins security: Manage Jenkins -> Configure Global Security
 - > ensure anonymous can trigger builds via webhook
4. Test: push a trivial change, watch webhook delivery in GitHub

Prevention: Monitor webhook success rate. Use GitHub webhook ping to test periodically.

Scenario 9: Pipeline Timeout

Symptom: Build aborted: "Timeout has been exceeded."

Cause: Build took longer than configured timeout. Usually a test or download hanging.

Your action:

1. Check which stage was running when timeout hit
2. If tests: check if a test is waiting for external service that's down
3. If download: Maven Central may be slow, or Nexus proxy is down
4. If deploy: Kubernetes pulling a large image can take time
5. Temporary fix: increase timeout in Jenkinsfile options block
6. Real fix: investigate why the stage took longer than expected

Prevention: Set reasonable timeouts. Monitor stage durations over time for drift.

Scenario 10: Docker Build Fails on Agent

Symptom: "Cannot connect to Docker daemon" or "docker: command not found" on agent.

Cause: Docker not installed on agent, Docker daemon stopped, jenkins user not in docker group.

Your action:

1. SSH into agent: which docker (is docker installed?)
2. `systemctl status docker` (is daemon running?)
3. `groups jenkins` (is jenkins user in docker group?)
4. If jenkins not in docker group: `sudo usermod -aG docker jenkins` && restart agent
5. If Docker disk full: `docker system prune -a`

Prevention: Include Docker in agent AMI/image. Monitor Docker disk usage.

Scenario 11: SonarQube/Nexus Connection Refused

Symptom: "Connection refused" to SonarQube (port 9000) or Nexus (port 8081).

Cause: SonarQube/Nexus server is down, restarting, or network issue.

Your action:

1. Verify the service is running: `ssh sonar-server, systemctl status sonarqube`
2. Check if port is listening: `netstat -tlnp | grep 9000`
3. Check service logs: `tail -f /opt/sonarqube/logs/sonar.log`
4. Common cause: SonarQube Elasticsearch ran out of memory -> increase JVM heap
5. If Nexus: check disk space. Nexus stops accepting uploads when disk is full.

Impact: ALL builds fail at the scan/upload stage. This is a priority escalation.

Scenario 12: "Works on My Machine" - Builds Fail on Jenkins

Symptom: Developer says "it builds fine locally" but fails on Jenkins.

Cause: Environment difference: Java version, Maven version, missing env vars, cached deps.

Your action:

1. Compare: Jenkins agent Java version vs developer's local Java
 - Agent: `java -version` on the agent
 - Developer: ask what version they use
2. Check environment variables: Jenkins may be missing something set locally
3. Check if Jenkins cleans workspace (should be clean build) while developer has cached state
4. Check if developer committed all files (sometimes .gitignore excludes needed files)

Key insight: Jenkins clean build is the FEATURE. It enforces reproducibility. If it fails on Jenkins, the local build has a hidden dependency on local state.

Scenario 13: Need to Rollback a Production Deployment

Symptom: Production deployment broke the service. Need to restore previous version.

Your action:

1. Identify the last known good version: check Nexus or Jenkins build history
2. For Kubernetes:
`helm rollback payment-service 12 -n production`
OR: `kubectl rollout undo deployment/payment-service`
3. For Tomcat:
 - a. Remove bad WAR from `/opt/tomcat9/webapps/`
 - b. Download good version from Nexus
 - c. Copy to webapps/ -> Tomcat hot-deploys
4. Verify rollback: `curl health endpoint`, check dashboards
5. Communicate: update incident channel, status page

Why this works: Every version is stored permanently in Nexus (WARs) or registry (images). Rollback = deploy an older version.

Scenario 14: Jenkins Controller Crashed

Symptom: Jenkins UI not accessible. http://:8080 returns nothing.

Your action:

1. SSH into Jenkins server: `sudo systemctl status jenkins`
2. If status is "failed" or "inactive": `sudo systemctl start jenkins`
3. Check why it crashed: `tail -100 /var/log/jenkins/jenkins.log`
Common: OutOfMemoryError, disk full, port conflict
4. If OOM: increase -Xmx in /etc/default/jenkins, then start
5. If disk full: free space (see Scenario 4), then start
6. If port conflict: `sudo netstat -tlnp | grep 8080` (who took the port?)

Impact: ALL builds stop. No new builds can be triggered. Running builds on agents may continue but cannot report back. This is a P1 incident.

Prevention: Jenkins high availability (HA) with standby controller. Regular health monitoring.

Scenario 15: Developers Requesting Pipeline Changes

Symptom: Developer wants to add a new stage, skip a stage, or change pipeline behavior.

Your action as L1:

1. If the change is to THEIR Jenkinsfile: they can PR it. Standard code review.
2. If the change affects the Shared Library: escalate to L2/L3. Admin-only access.
3. If they want to skip a security stage: DO NOT bypass. Route to security team.
4. If they need a new Jenkins plugin: document the request, escalate to L2.
5. If they need new credentials added: verify with the team that owns the credential.

Key principle: L1 does not make structural pipeline changes. L1 operates, monitors, and routes.

PART 4

Jenkins Administration

Backup Strategy (Critical)

If Jenkins controller dies and you have no backup, you lose ALL job configurations, build history, credentials, and plugin settings. Recovery is manual and takes days.

```
# What to back up: /var/lib/jenkins/config.xml # global config /var/lib/jenkins/jobs/ # all job
configs + history /var/lib/jenkins/users/ # user accounts /var/lib/jenkins/secrets/ # encrypted
credentials /var/lib/jenkins/nodes/ # agent configs /var/lib/jenkins/plugins/ # installed
plugins # Simple backup script (run via cron daily): #!/bin/bash
BACKUP_DIR="/backups/jenkins/$(date +%Y%m%d)" mkdir -p $BACKUP_DIR tar czf
$BACKUP_DIR/jenkins-home.tar.gz \ --exclude='*/workspace' \ --exclude='*/builds/*/archive' \
/var/lib/jenkins/ aws s3 cp $BACKUP_DIR/jenkins-home.tar.gz s3://company-backups/jenkins/ find
/backups/jenkins -mtime +7 -delete # keep 7 days locally
```

Jenkins Security & RBAC

Enterprises like Citizens Bank require strict access control. Not everyone should be able to configure jobs or view credentials.

Feature	What It Controls	How to Configure
Matrix Authorization	Per-user permissions (read, build, configure, admin)	Manage Jenkins -> Configure Global Security -> Matrix-based
Folder Permissions	Per-team access (Team A sees only their jobs)	Install Folders + Role-based Authorization Strategy plugins
Credentials Scoping	Credentials visible only to specific folders/jobs	Add credential at folder level, not global
Agent-to-Controller Security	Prevent agents from accessing controller filesystem	Manage Jenkins -> Security -> Agent-to-Controller Access Control
Audit Trail	Log who did what (build, config change, credential access)	Install Audit Trail plugin -> sends to Splunk/S3
CSRF Protection	Prevent cross-site request forgery attacks	Manage Jenkins -> Security -> Enable CSRF (on by default)

Monitoring Jenkins Itself

Jenkins is infrastructure. Monitor it like any other critical service.

Metric	What to Watch	Alert Threshold
JVM Heap Usage	Memory pressure on controller	Alert at > 80%

Build Queue Length	Jobs waiting to run	Alert at > 50 for > 10 min
Agent Count	Online vs configured agents	Alert when any agent offline > 5 min
Build Duration	Stage timing drift (getting slower?)	Alert when 2x normal duration
Disk Usage	Controller and agent disk space	Alert at > 80%
Failed Builds Rate	Percentage of builds failing	Alert when > 30% failures in 1 hour
Plugin Health	Plugin loading errors	Alert on any error in jenkins.log

```
# Prometheus monitoring for Jenkins: # Install "Prometheus Metrics" plugin in Jenkins # Jenkins
exposes metrics at: http://jenkins:8080/prometheus/ # Configure Prometheus to scrape this
endpoint # Build Grafana dashboard with key metrics above
```

Essential Jenkins Plugins (Enterprise)

Plugin	What It Does	Why Essential
Pipeline	Read Jenkinsfile from repos	Core pipeline functionality
Git	Clone from GitHub/GitLab	Source code management
GitHub / GitHub Branch Source	Webhooks + multi-branch pipelines	Auto-discover branches
Credentials	Store secrets securely	Never hardcode passwords
Docker Pipeline	Build/push Docker images	Container integration
Kubernetes	Dynamic pod agents	Enterprise scaling
SonarQube Scanner	Integrate SonarQube analysis	Code quality gates
OWASP Dependency Check	Security dependency scanning	DevSecOps requirement
Role-based Authorization Strategy	RBAC per folder/job	Enterprise security
Blue Ocean	Modern pipeline visualization UI	Better user experience
Slack Notification	Send alerts to Slack	Team communication
Prometheus Metrics	Expose Jenkins metrics	Monitoring integration
Job DSL / Configuration as Code (JCasc)	Define Jenkins config in YAML files	Infrastructure as Code for Jenkins itself
Pipeline Utility Steps	File operations, JSON/YAML parsing in pipeline	Common pipeline needs
Audit Trail	Log who did what	Compliance requirement
Timestampper	Add timestamps to console output	Debug timing issues

Build Discarder	Auto-delete old builds	Prevent disk issues
-----------------	------------------------	---------------------

Jenkins Configuration as Code (JCasC)

Modern Jenkins can be configured entirely via YAML files instead of clicking in the UI. This is Infrastructure as Code for Jenkins itself.

```
# jenkins.yaml (JCasC configuration file)
jenkins:
  systemMessage: "Jenkins configured via JCasC"
  numExecutors: 0 # controller runs 0 builds
  securityRealm: local:
    users:
      - id: "admin"
        password: "${ADMIN_PASSWORD}"
    authorizationStrategy: roleBased:
      roles:
        global:
          - name: "admin"
            permissions: ["Overall/Administer"]
          - name: "developer"
            permissions: ["Job/Read", "Job/Build"]
  clouds:
    - kubernetes:
        name: "k8s"
        serverUrl: "https://kubernetes.default"
        namespace: "jenkins"
  jenkinsUrl: "http://jenkins:8080"
  podLabels:
    - key: "jenkins/agent"
      value: "true"
  tool:
    maven:
      installations:
        - name: "Maven-3.9"
      properties:
        - installSource: installers:
            - maven:
                id: "3.9.6"
```

ENTERPRISE PATTERN: JCasC is how enterprises manage Jenkins at scale. Instead of one person clicking settings in the UI, the configuration is in a Git repo, version controlled, and applied automatically. If Jenkins controller dies, rebuild it from JCasC in minutes.

PART 5

Interview Preparation

Jenkins Interview Questions

35 questions from basic to advanced, with production-grade answers

Architecture & Fundamentals

Q1. What is Jenkins?

Jenkins is an open-source CI/CD automation server written in Java. It runs as a system service on a server (port 8080), reads pipeline definitions from Jenkinsfiles in Git repos, and executes build/test/deploy stages on worker agents. It stores build history, manages credentials, and integrates with tools like Maven, Docker, SonarQube, and Kubernetes.

Q2. What is the difference between the Jenkins controller and an agent?

The controller (master) manages the UI, build queue, scheduling, and build history. It does NOT run builds - that is a security and performance risk. Agents are worker machines that run actual build commands. Each agent has executors (slots for concurrent builds). More agents = more parallel capacity.

Q3. What is a Jenkins executor?

An executor is a slot on an agent that can run one build at a time. An agent with 4 executors can run 4 builds concurrently. The total concurrent capacity = agents x executors. Too many executors per agent causes resource contention; too few wastes agent capacity.

Q4. How does the Jenkins build queue work?

When a build is triggered, it enters the controller's queue. The controller finds a free executor on an agent that matches the job's label requirements. If no executor is free, the job waits. Queue management includes: priority sorting, throttling concurrent builds, and matching agents by label.

Q5. What is the difference between Freestyle and Pipeline jobs?

Freestyle: configured through Jenkins UI, not version-controlled, limited to linear steps. Legacy approach. Pipeline: defined in a Jenkinsfile in the Git repo, version-controlled, supports multi-stage flows, parallel execution, conditionals, and approval gates. Always use Pipeline for new work.

Multi-Agent & Scaling

Q1. How would you speed up Jenkins if 400 jobs are queued with 10 agents?

Immediate: check for offline agents and stuck builds, increase executors on existing agents. Short-term: add more static agents. Proper fix: implement dynamic agents using the Kubernetes plugin or EC2 plugin for auto-scaling. Also: run pipeline stages in parallel, cache Maven dependencies, skip unnecessary stages on feature branches, and enable Priority Sorter for critical jobs.

Q2. What are Jenkins dynamic agents?

Agents created on demand and destroyed after use. Three types: Kubernetes pods (fastest, ~5s startup), EC2 instances (~60s startup), Docker containers (~2s startup). The Kubernetes plugin is the enterprise standard - creates a pod per build, destroys it after. No idle cost. Unlimited scaling based on cluster capacity.

Q3. What are Jenkins agent labels and how do you use them?

Labels are tags assigned to agents: "linux", "docker", "java", "gpu". In the Jenkinsfile, agent { label 'linux && docker' } routes the build to an agent with both labels. This ensures builds run on appropriate hardware - Maven builds go to agents with Maven, Docker builds go to agents with Docker daemon.

Q4. How do you configure a Kubernetes-based Jenkins agent?

Install the Kubernetes plugin. Configure: Manage Jenkins -> Configure Clouds -> Kubernetes. Set: cluster URL, namespace, pod templates. In Jenkinsfile: agent { kubernetes { yaml "pod spec here" } }. Each build creates a pod with specified containers. Pod is deleted after build completes.

Q5. What is the difference between a permanent agent and a cloud agent?

Permanent: always on, fixed EC2/VM, registered manually. Wastes money when idle. Cloud: created on demand when builds are queued, destroyed when idle. Scales automatically. Enterprise choice: cloud agents (K8s or EC2 plugin) for build workloads, permanent agents only for special hardware.

Pipeline & Jenkinsfile

Q1. What is a Jenkinsfile?

A text file named "Jenkinsfile" at the root of a Git repository that defines the CI/CD pipeline in Groovy DSL. It is pipeline-as-code - version controlled, reviewable via PRs, and auditable. Jenkins reads it when the pipeline runs.

Q2. What is the difference between Declarative and Scripted pipeline?

Declarative uses structured blocks (pipeline{}, stages{}, stage{}) and is the recommended approach - readable by anyone. Scripted uses free-form Groovy (node{}, stage{}) for complex logic but is harder to read. Enterprise standard is Declarative with script{} blocks when needed.

Q3. What does "checkout scm" do?

It tells Jenkins to clone or fetch the source code from whatever SCM (Git repository) the pipeline job is configured to use. It is automatic - the repo URL comes from the job configuration, not hardcoded in the Jenkinsfile.

Q4. How do you run stages in parallel?

Wrap stages in a parallel block: stage('Checks') { parallel { stage('Tests'){...} stage('Sonar'){...} } }. Both run simultaneously. Pipeline waits for all to complete. Reduces total time significantly.

Q5. What is the "when" directive in Jenkins?

Conditional stage execution. Common uses: `when { branch 'main' }` (only on main), `when { expression { return params.DEPLOY } }` (based on parameter), `when { changeset "src/**" }` (only if certain files changed). Prevents unnecessary stages.

Q6. How do you implement approval gates in Jenkins?

Use the input directive: `input { message 'Deploy to production?' ok 'Yes' submitter 'managers' }`. Pipeline pauses, shows approval button in Jenkins UI. Only specified users can approve. Essential for regulated environments like banks.

Q7. What is the post block in a Jenkinsfile?

Actions that run after all stages complete. Sections: `always{}` (cleanup), `success{}` (notifications), `failure{}` (alerts), `unstable{}` (test failures), `aborted{}` (manual cancel). Used for workspace cleanup, Slack notifications, email alerts.

Q8. What are Jenkins environment variables?

Variables available in pipeline: `env.BUILD_NUMBER` (auto), `env.BRANCH_NAME` (auto), custom via `environment{}` block, secrets via `credentials('id')`. Access in shell: `${BUILD_NUMBER}`. In Groovy: `env.BUILD_NUMBER`.

Administration & Troubleshooting

Q1. A build is stuck for hours. How do you investigate and fix it?

Check Console Output for the last line. If "Waiting for user input" - someone needs to approve. If output stopped - process is hung. Kill it via the red X button. If that fails, use Script Console. Check if agent is alive. Check for network timeouts. Add `timeout()` option to prevent recurrence.

Q2. Jenkins disk is full. What do you do?

SSH in, run `df -h` and `du -sh /var/lib/jenkins/*`. Delete old workspaces, Docker images, build artifacts. Enable build discarder on all jobs (keep last 20). Configure Workspace Cleanup plugin. Move to larger EBS volume if persistent issue. Monitor disk with alerts at 80%.

Q3. How do you back up Jenkins?

Back up `/var/lib/jenkins/` (excluding workspace/) to S3 daily via cron script. Include: `config.xml`, `jobs/`, `users/`, `secrets/`, `plugins/`, `nodes/`. Encrypt backups (secrets/ contains credentials). Test restore procedure quarterly. For enterprise: use Jenkins Configuration as Code (JCasC) so config is in Git.

Q4. How do you manage credentials securely in Jenkins?

Store in Jenkins Credentials store (encrypted at rest). Reference by ID in Jenkinsfile using `credentials()` or `withCredentials()`. Never hardcode. Jenkins masks credential values in console output. Scope credentials to folders for least-privilege. Rotate credentials on schedule.

Q5. What is Jenkins Configuration as Code (JCasC)?

A plugin that lets you define Jenkins configuration in a YAML file instead of the UI. Covers: security settings, tools, clouds, credentials, node configuration. Benefits: version controlled, reproducible, disaster recovery in minutes. Enterprise standard for managing Jenkins at scale.

Q6. How do you monitor Jenkins health?

Install Prometheus Metrics plugin. Jenkins exposes metrics at /prometheus/. Monitor: JVM heap, build queue length, agent count, build duration, failed build rate, disk usage. Alert thresholds: heap > 80%, queue > 50 for > 10 min, any agent offline > 5 min.

Q7. What is a Jenkins Shared Library?

A separate Git repo containing reusable pipeline code. Structure: vars/ for callable functions, src/ for classes. Any Jenkinsfile can import it with @Library. Benefits: DRY pipelines across 50+ repos, security enforcement (admin-only write access), one update applies everywhere.

Enterprise & Security

Q1. How do you ensure developers cannot bypass security scans?

Use Jenkins Shared Libraries. Security stages (SonarQube, OWASP) are defined in a centrally-controlled library repo with admin-only write access. Developers call library functions in their Jenkinsfile but cannot modify the security stages. If they try to skip a stage, they cannot - it is baked into the library function.

Q2. How does Jenkins integrate with change management?

For regulated environments, the Deploy stage includes an "input" step requiring approval from authorized personnel. Audit Trail plugin logs who approved. ServiceNow integration can automatically create change records. Build metadata (commit, approver, scan results) is attached to the change ticket.

Q3. What is the difference between Jenkins Master/Slave and Controller/Agent?

Same thing, renamed. Master was renamed to Controller, Slave was renamed to Agent in Jenkins 2.319 (2021) for inclusive terminology. Functionally identical. Use the new terminology in interviews.

Q4. How do you handle Jenkins upgrades in production?

Never upgrade production Jenkins directly. Process: 1. Test upgrade on staging Jenkins first. 2. Back up production Jenkins. 3. Schedule maintenance window. 4. Upgrade Jenkins + plugins. 5. Verify all pipelines still work. 6. Monitor for 24 hours. Rollback plan: restore from backup.

Q5. What is a Jenkins multibranch pipeline?

A job type that automatically discovers all branches in a Git repo and creates a pipeline for each. When a new branch is pushed, Jenkins auto-creates a pipeline. When deleted, Jenkins removes it. Combined with GitHub Branch Source plugin, this is how enterprises manage pipelines for dozens of branches.

The Jenkins Commands Cheat Sheet

```
# Service management sudo systemctl start jenkins sudo systemctl stop jenkins sudo systemctl
restart jenkins sudo systemctl status jenkins sudo systemctl enable jenkins # auto-start on
boot # First-time setup sudo cat /var/lib/jenkins/secrets/initialAdminPassword # Logs tail -f
/var/log/jenkins/jenkins.log # Disk usage du -sh /var/lib/jenkins/* | sort -rh | head df -h #
Backup tar czf jenkins-backup.tar.gz --exclude='workspace' /var/lib/jenkins/ # Kill stuck build
(Script Console: http://jenkins:8080/script)
Jenkins.instance.getItemByFullName("job-name").getBuildByNumber(123).doStop() # List all jobs
(Script Console) Jenkins.instance.getAllItems(Job.class).each { println it.name } # Check agent
status (Script Console) Jenkins.instance.computers.each { println "${it.name}:
${it.isOnline()}" } # Restart Jenkins safely (waits for running builds to finish)
http://jenkins:8080/safeRestart # Key directories /var/lib/jenkins/ # Jenkins home
/var/lib/jenkins/workspace/ # build workspaces /var/lib/jenkins/jobs/ # job configs + history
/var/lib/jenkins/secrets/ # encrypted credentials /var/lib/jenkins/plugins/ # installed plugins
/var/log/jenkins/jenkins.log # application log /etc/default/jenkins # JVM options (heap size)
```

DAILY REFERENCE: Print this cheat sheet and keep it at your desk. These are the commands you will use every day as a Jenkins administrator.

End of Jenkins Guide

You now have the complete Jenkins knowledge from beginner to advanced. The architecture diagram on page 4 is the foundation. The 400 jobs answer on page 8 is what failed you last time. The 15 production scenarios are what real work looks like. The 35 interview questions cover what Citizens Bank will ask.

Next step: build a Jenkins server with 3 agents. Run 50 builds. Break it. Fix it. That is how this knowledge becomes permanent.